CSC274 - Web Proxy Server

Technical Report

Hussein Abadi
Jinan Al Mokdad
Serge Sevadjian

Table of Contents

1. Explanation of the Internal and External Modules
2. Overall System Workflow
3. Caching Strategy
4. Challenges Faced
5. Testing Methodology
6. Member Contributions

This document explains the design of the program and its files, each module used, the testing methodology, and each member's contribution to the project.

The code of the project is on Github here and the commit history is visible. After discussing with each other on how to approach the project, each of us did Level 1 on our own starting April 5 and then we settled on one of the versions and split the remaining levels and tasks within each level among ourselves.

Explanation of Internal and External Modules

Internal Modules

In total, the number of functionalities and classes created by us in separate Python modules are 8 (ordered from high priority to least):

Module	Description
proxy.py	This is the main Python file that runs the proxy server. It has three methods: 1. start_proxy: starts the proxy server by creating a socket for the server, binding it to a port, and waiting to accept a client's connection. Before accepting a client's connection, it checks if the client's IP address is blacklisted; if yes, it blocks the request. 2. handle_client: this method is used by start_proxy() whenever a client connection is accepted by the server. It receives the client's request from its socket, parses the request, checks for errors or forbidden requests (if the request is a non-supported method, or if the host or the port trying to connect to is blocked), and handles the request depending on the method (CONNECT for HTTPS or GET for HTTP). 3. fetch_from_server: this method is used by handle_client if the request is GET (HTTP) and a cache miss occurs. It opens a new socket for the target server that the client is requesting to load the page from, it connects to the server based on the host and the port that the client is trying to connect to, it will prepare a forward request for the target server and it will send it. Then it will take the chunks of data sent from the target server and send it to the client's socket till the data received from the server is empty (no more data left). Once done, it caches the full response content and closes the target server's socket.
ProxyCache.py	This class is used to cache the HTTP responses that the server gets from the target servers, only for GET requests. We used OrderedDict to store the data in such a format: url => {"data": bytes, "timestamp": float}. We specifically used OrderedDict because it has the methods .move_to_end(key) and .popitem(last=False) which suited our purpose for the LRU algorithm implementation. Whenever a cache hit occurs, the URL's data that was retrieved from the cache moves to the end of the dictionary. So the ones that are at the beginning would be, most of the

Module	Description
	time, the least requested URLs because they aren't moved to the end. Hence, when a new URL's response is going to be cached and the dictionary's size is full, it uses the .popitem(last=False) method to remove the first cached URL. We set the maximum size of the dictionary to 10.
CacheStats.py	This class is used to store data on the total cache hits, total cache misses, total hit time, and the total miss time. Everytime these occur in proxy.py (GET HTTP only, not CONNECT) it will store the information in this class (as proxy.py initializes an object for this class already), and at the end of handling a client's request it will print a performance summary. The summary includes: - The number of total requests - The number of total hits - The number of total misses - The average miss time (total hit time / the number of total hits) - The average hit time (total miss time / the number of total misses) - Speed factor: the average miss time / the average hit time
filtering.py	This Python file has methods that are used by proxy.py to prevent requests to blacklisted hosts and ports or from blacklisted client IP addresses. For fast and convenient modification, we used .txt files in the blacklists folder to store the three different blacklists in three different files: addresses.txt, hosts.txt, and ports.txt. In order not to load the text files every time the server needs to check if a host/port/client IP is blacklisted, we are tracking the last modified date of each of the files. It only loads from the text file(s) if the file hasn't been modified after the stored last modified date, or else it uses the existing blacklisted entries stored in the set.
http_parser	This Python file has methods used by proxy.py to properly parse requests and responses. - For requests: it returns the method, host, port, path, and remaining headers. If the method is not supported (anything other than GET and CONNECT) or if an error occurred when trying to parse, it returns None. - For response: It returns the status code and the status text. If the length of the response header line is not greater than or equal to 2, that means it's not correct and it returns Unknown with an empty string for status text.
http_tunneling	This Python file has methods used for CONNECT (HTTPS) requests. First, it opens a raw TCP connection to the target server based on the host and port. Then it sends a 200 Connection Established response to the client's socket, so the browser knows that a TCP connection has been established. Then the server opens two threads: one from the client to the server, and another from the server to the client. This is essential because it's full-duplex. Both, the client and the target server, send data to each other until no more data is left to be sent from the client to the server and vice versa. The proxy is only playing the role of opening the TCP connection between the client and the server so they can talk to each other; the messages exchanged are encrypted (HTTPS), and our proxy cannot cache them as

Module	Description
	it would not be legitimate.
logger	This Python file has methods to log the following actions: requests received, responses sent, rejected methods, forwarding the request to target server, receiving a response from the target server, sending the response back to the client, establishing a tunnel connection with the target server (CONNECT), closing a tunnel connection, blocking a host/address/port, cache hit/miss, when the least entry in the cache was removed and if a cache was expired.
send_errors	This small Python file has a method for formatting a proper 403 forbidden response message, and a general error sending method that will format a proper response message based on the status code and the text the server wants to show to the client. Both of them send the responses to the client's socket.

External Modules

These are the external modules we imported within the Python modules mentioned above:

Module	Description
threading	Allows you to run multiple tasks concurrently by sharing the same memory space.
time/datetime	Handles time primarily through the built-in time module, which focuses on system-level time and timestamps.
OrderedDict	A dictionary subclass from the collections module that remembers the exact order in which items were inserted (helped in implementing LRU algorithm for caching).
socket	The foundation for network communication in Python. It allows programs to talk across a network by creating endpoints (sockets) that send or receive data
colorama	Allows you to print colored text in the terminal. Needs to be installed using PIP, found here.
datetime	The standard way to work with dates and times. It provides classes to create, manipulate, and format time-related data.
os	A built-in library that allows your code to interact with the underlying operating system. It provides a portable way to use system-dependent functionality like navigating files, managing processes, and accessing environment variables.
re	A built-in library used for Regular Expressions (RegEx). It allows you to

Module	Description
	search, match, and manipulate strings based on complex patterns rather than just fixed text.
pathlib.Path	Provides an object-oriented way to handle file system paths, replacing older string-based modules like `os.path` . It treats paths as objects with methods and attributes, making code more readable and cross-platform

Overall System Workflow

This section explains how the proxy server processes a client request from the moment a connection is established until the response is returned.

1. Client Connection

The process begins when a client (such as a web browser/curl) connects to the proxy server. The server, running in proxy.py, listens on a specified port (we chose 10000) and accepts incoming connections. For each new client connection, a separate thread is created to allow multiple clients to be handled concurrently.

2. Receiving and Parsing the Request

Once connected, the proxy receives the client's request through the socket. The request is then passed to the parsing module http_parser.py, which extracts key components:

- HTTP method (only GET or CONNECT accepted)
- Target host
- Port number
- Requested path
- Any additional headers

If the request is malformed or uses an unsupported method, an appropriate error response is sent back to the client.

3. Filtering and Access Control

Before processing the request further, the proxy checks whether it should be blocked. The filtering module filtering.py verifies:

- If the client's IP address is blacklisted
- If the requested host is blocked
- If the target port is restricted

If any of these conditions are met, the proxy immediately responds with a 403 Forbidden message and terminates the request.

4. Handling HTTPS Requests (CONNECT Method)

If the request method is CONNECT, the proxy establishes a tunnel between the client and the target server using the HTTPS tunneling module.

- A TCP connection is opened to the target server
- The proxy sends a “200 Connection Established” response to the client
- Two threads are created to relay data in both directions (client - server + server - client)

At this stage, the proxy acts only as a relay. Since the data is encrypted, it cannot be inspected or cached. After a preset timeout, the tunnel closes, ending the connection between the client and the target server.

5. Handling HTTP Requests (GET Method)

If the request method is GET, the proxy proceeds with cache handling. The proxy checks whether the requested resource is already stored in the cache ProxyCache.py:

- **Cache Hit:**
  The cached response is retrieved and immediately sent back to the client. The cache entry is also updated to reflect recent usage (LRU).
- **Cache Miss:**
  The proxy proceeds to fetch the resource from the target server.

6. Fetching Data from the Target Server

For a cache miss, the proxy:

- Opens a new socket connection to the target server
- Constructs and forwards the HTTP request
- Receives the response in chunks
- Streams the response back to the client as it is being received

Once the full response is received, it is stored in the cache for future use.

7. Caching the Response

The received response is stored in the cache along with a timestamp. The cache follows an LRU (Least Recently Used) policy:

- Frequently accessed items are kept
- Older or less-used entries are removed when the cache reaches its maximum size
- Expired entries are also removed based on a predefined timeout (we chose 60 seconds)

8. Logging and Performance Tracking

Throughout the entire process, the proxy logs all major events using the logging module logger.py (both in the console and saves to an external text file), including:

- Incoming requests
- Blocked requests
- Cache hits and misses
- Forwarded requests and received responses

At the same time, performance statistics are updated using CacheStats.py, which tracks metrics such as:

- Total cache hits and misses
- Average response times
- Cache efficiency (speedup factor)

9. Closing the Connection

After the response is fully sent to the client, the connection is closed. The thread handling the request terminates, freeing resources for future connections.

Caching Strategy

As previously explained, the caching strategy we used is the LRU algorithm with a cache timeout of 60 seconds. They're stored in the OrderedDict dictionary with the following key-value pair structure:

```
url: {data, timestamp}
```

We implemented cache only for HTTP GET requests. Our proxy server's job is to open a full duplex connection between the client and the target server, and all transmissions are done

between them and our proxy does not see the unencrypted exchanged messages. Even if there are workarounds to this, they would be a security compromise and unethical.

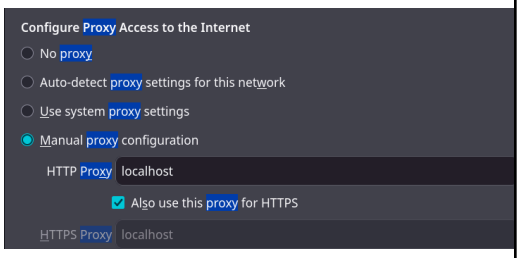
Challenges Faced

One challenge we faced was deciding whether to implement a GUI interface or not. Due to the time constraint, we didn't want to rush into it so we would have time to write this report document and review the code. We also implemented and tweaked the following instead of doing a GUI:

- We reformatted the logging texts.
- We implemented terminal coloring using the colorama library so the terminal prints would be more appealing to look at. Different elements have their own colors, such as the timestamps, errors, and CONNECT related loggings.
- We implemented auto refresh for the blacklist text files based on the last modified date. This would be more convenient than restraining the server just to update the blacklisted hosts/ports/client IPs.

Testing Methodology

We used two testing methodologies: curl and Firefox's proxy settings.

curl	Firefox
We used the following command for all of our curl testings through a terminal/CMD: `curl -v –proxy http://localhost:10000 https://google.com` The https://google.com URL can be replaced with any other URL	In order to use the proxy server in Firefox, we first configured it from browser settings. 1. Open the application menu 2. Click on Settings 3. Search for "proxy" 4. Go to "Configure proxy" 5. Select "Manual proxy configuration" 6. Set the HTTP Proxy to "localhost" and the Port to "10000", also check the "Also use this proxy for HTTPS" checkbox  7. Click OK After this, any attempt to access a website

	will be done through our proxy.

Notes:

- If the port number was changed in the code, ensure in both of the above you adjust the port number as well.
- Ensure that the server is actually running, or else (critically in Firefox's case) no website will be able to load.
- If the blacklist hosts file was updated, and the website is still loading in the browser, clear the cache of that specific website and then try again. We faced this issue where the browser was loading a blacklisted website because it had it cached from a previous response, and after clearing the cache of that specific website the proxy blocked it.

Member Contributions

Member	**Levels**	**Description**
Hussein Abadi	1, 4.B, 5.B, 5.C	Persistent logging in files, smart caching and performance summary
Jinan Al Mokdad	1, 2.B, 3.B	Improved logging with timestamps, simple caching policy (timeout and max cache size)
Serge Sevadjian	1, 2.A, 3.A, 4.A, 5.D, colorama	Threading, basic caching, blacklist filtering, HTTPS tunneling, terminal coloring

Notes:

- For Level 1, we all did our own versions first and then we settled and built on top of Hussein's.
- All of us did small bug fixes, but we did not include them in the table. The specific bug fixes can be seen in the commit history on the Github page of the project.

Thank you.